D3 - Final engineered product

Project name
ANDREAS

Agreement Number

Duration
M9 - M10

Deliverable reference
D3

Date of delivery
M10

Responsible partner
E4

Editor and authors
Marco Cicala(E4), Federica Filippini (PoliMI), Fabrizio Magugliani (E4), Maciej Riedl (7bulls), Pawel Skrzypek (7bulls)

Reviewers
Danilo Ardagna (PoliMI)
Katarzyna Materka (7bulls)

Distribution
Confidential

Executive Summary

This deliverable describes the third and final release of the GPU scheduler developed within ANDREAS which has been integrated (within T3) into the production process via the formulation of a supply chain, manufacturing process, and time plan. Training material is created for the staff assembling the product and for the field support staff. The product life cycle, including a multi-year support & update plan, is developed. The sales team (Sales and Pre-Sales representatives, Marketing executives) at 7BULLS and E4 is provided with the information about the product, its features and the profile of the target prospects.

This deliverable is the results of the work achieved within task *T2 - EUBra-BigSEA scheduler integration with the SLURM queue,* and *T3 - Bring to Market the integrated solution* during the last four months of the project.

The final results have shown how the scheduling policies are effective providing significant energy cost reduction (in the range 86% - 94%) and speedup of the applications executions (between 3x and 5x on average).

D3 - Final engineered product **1**
Executive Summary 1
1. Introduction **4**
2. Technical changes in 3rd release **5**
3. Jobs optimizer changes in 3rd release **9**
3.1 Jobs Optimizer API 9
3.2 Code structure 10
4. Testing results **13**
4.1 Applications understudy 13
4.2 Profiling results 13
4.3 Exploratory analysis 16
4.4 Validation in the ARMIDA setup 19
5. User guide **23**
5.1 Jobs Manager configuration file 23
5.2 How to prepare setup folders 24
5.3 How to prepare the scripts to be launched 24
5.4 How to start the application 26
5.5 How to manage running applications 26
6. Business Plan **29**
7. Conclusion **30**
8. References **31**
Appendix A/Business Plan **32**
Executive Summary 33
Product 33
Customers 33
Partners' Description 33
The Mission 33
The Vision 34
Politecnico di Milano 34
7bulls.com 34
Market Research 35
Machine Learning 35
Detailed Description of Customers 35
Product's Advantages 36
Product and Support Plan 36
Product 36

Intellectual Property Rights 37
Implementation Property Rights 38
Pricing Structure 39
Licensing model for ANDREAS: 39
T&C 39
Revenue split 40
Customer Support/Field Support 40
Product Lifecycle 40
Research and Development 40
Marketing & Sales 41
Go To Market Strategy 41
Growth Strategy 41
Communicate with the Customer 41
Sales channels 41
Forecast (as of March 2021) 42

1. Introduction

This deliverable describes the third and final release of the GPU scheduler developed within the ANDREAS project and includes the changes introduced in the third release compared to the second release (described in deliverable D2). This deliverable is the results of the work achieved within task *T2 - EUBra-BigSEA scheduler integration with the SLURM queue* and *T3 - Bring to Market the integrated solution* during the last four months of the project.

This document is organized as follows. The next section (Section 2) introduces the technical changes applied in the 3rd release in the high level architecture of the ANDREAS scheduler. Section 3 describes the changes applied specifically in the 3rd release to the Job Optimizer. Section 4 reports the results obtained during the experimental campaigns for the 3rd release of ANDREAS, highlighting the fact that the results provided by the ANDREAS Optimizer have been evaluated in the ARMIDA setup. Section 5 contains the Users' Guide and describes how to prepare the jobs and use the ANDREAS framework. Section 6 provides a description of the considerations and conditions taken into account in the development of the Business Plan, which is reported in Appendix A. Section 7 draws the Conclusions of the work performed within the projects and the results achieved.

2. Technical changes in 3rd release

This section presents the final high-level architecture of the solution created within the ANDREAS project, along with the changes compared to architecture described in deliverable "D2 - Testing and 2nd release of the integrated scheduler " [6]. The key changes provided in the last release of the platform are the ability to stop and restart jobs from a snapshot and migrating jobs across different GPUs, based on the results of the optimization.

Jobs are python Machine Learning scripts prepared by the user. They need to use Tensorflow features for GPU usage. It is also important to prepare a dedicated directory for each job, containing input files. Trained models (or snapshots) will be stored there as well. Each job will be launched in a safe docker environment. Though it is possible to use a custom docker image, we highly encourage using the one prepared specifically for ARMIDA (tf2-gpu.aarch64:v1.2). It contains multiple libraries needed for deep learning training workloads like Keras or Numpy. Example scripts used during the development are also provided and include multiple layers neural networks such as EfficientNet network [7], convolutional networks [6] and LSTM networks [5]. More detailed instructions on how to write scripts are available in Section 5.1.

In Figure 1 the final architecture of the system is presented.

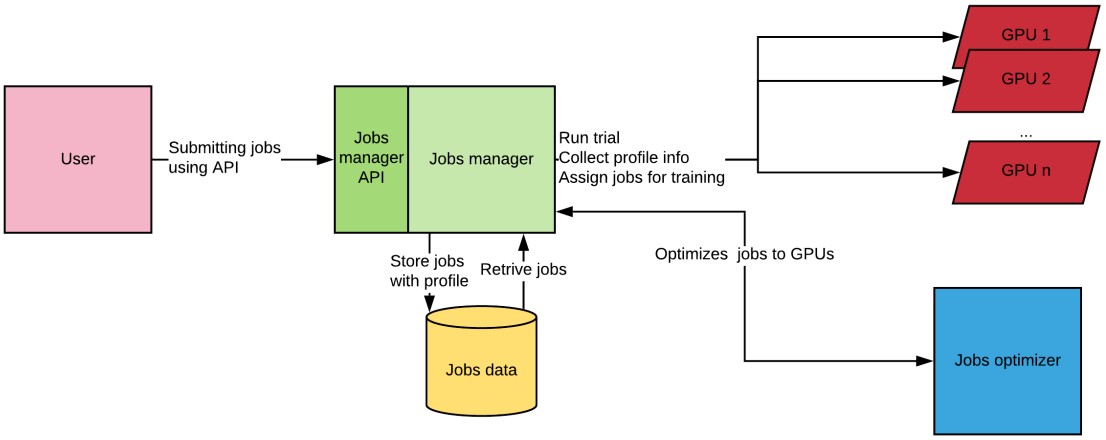


Figure 1. Final ANDREAS high level architecture diagram

In Table 1, the final list of components of the system are reported.

Id	Component	Description	Type
1.	Jobs manager API	The API exposed by Jobs manager component to allow jobs submission.	Internal ANDREAS component
2.	Jobs manager	The component responsible for handling the workflow in ANDREAS.	Internal ANDREAS component
3.	Jobs optimizer	The component responsible for jobs placement optimization.	Internal ANDREAS component
4.	Jobs data	Database to store jobs and profile information.	Internal ANDREAS component
5.	GPU	SLURM handled GPUs used to collect profile information and execute training.	External

Table 1. Final list and description of ANDREAS components

The sequence diagram of the final implemented solution is depicted in Figure 2 and described below. Another important change with respect to the 2nd release is the usage of Jobs manager API to submit the job to the ANDREAS system, instead of using the SLURM queue, as initially planned. Thanks to that, the jobs can be submitted for scheduling independently and our solution is not tightly coupled to SLURM.

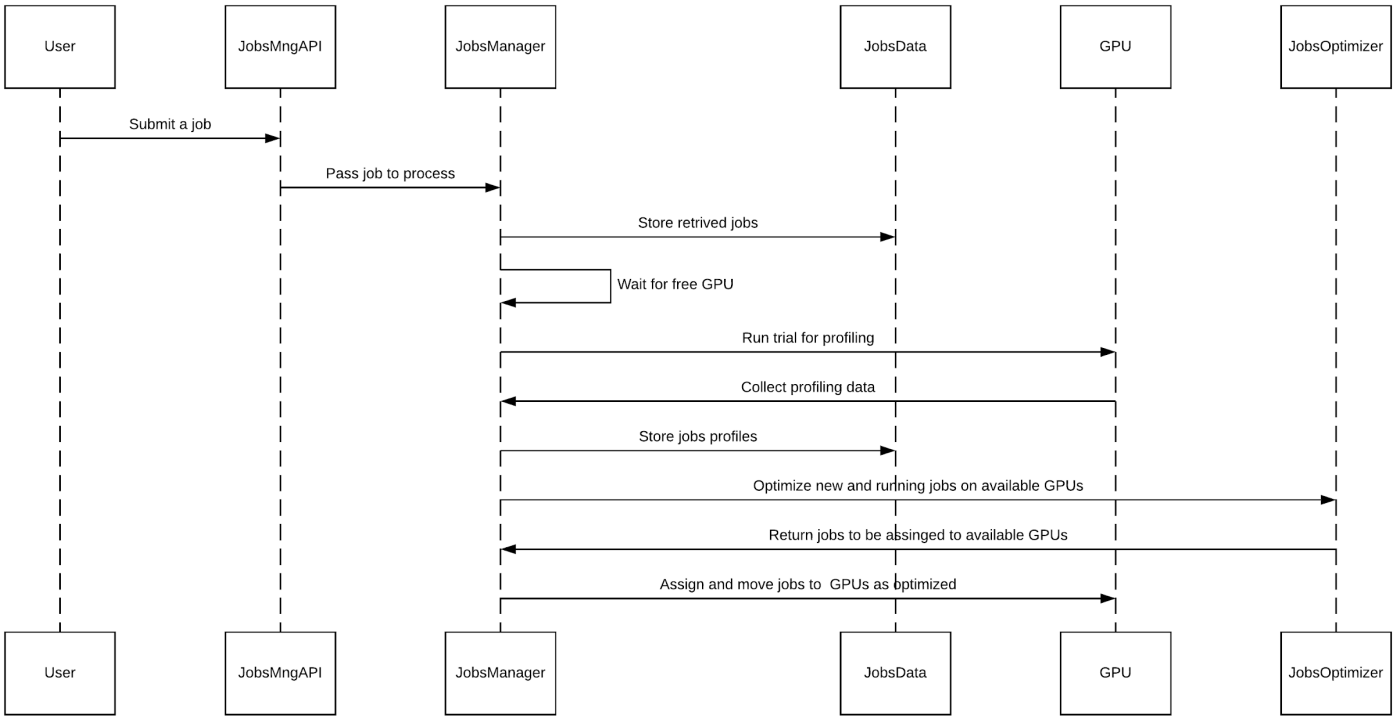


Figure 2. Final ANDREAS sequence flow diagram

The actual flow is reported below:

1. The user provides connected nodes' names using Jobs Manager API. It also needs to specify which nodes are dedicated for profiling. These nodes setup can be changed anytime.
2. A new machine learning training Job is submitted using the Jobs Manager API.
3. The Jobs Manager stores Job-related information in the Jobs Data database.
4. The Jobs Manager is waiting for the available GPU to start profiling operations.
5. The Jobs Manager is running Jobs to gather profiling information - according to the profiling rules (currently per epoch execution times are gathered by running few training epochs on a dedicated profiling node). If a new job is submitted, that was already profiled in the past, the Jobs Manager will start profiling with a different setup than in previous profilings, possibly using different GPU types or increasing the GPU number.
6. The Jobs Manager stores profiling data in the JobsData database.
7. The Jobs Manager invokes the Jobs Optimizer to identify the optimal Jobs execution setup.
8. The Optimizer identifies the optimal Jobs scheduling providing the number, type of GPU to be used to run the job within its deadline, minimizing energy consumption and job tardiness in case the deadline cannot be fulfilled. The Optimizer also identifies the node of the cluster where the Job (application) container has to be run.
9. The Jobs Manager runs the Job according to the optimization results, moving the jobs in progress among different GPUs if necessary .

10. Steps 7-9 are repeated periodically so that the optimal jobs assignment is identified in any point in time.

The flow described above has been implemented in the form of the final ANDREAS release and was tested with benchmarking applications (see also Section 4).

3. Jobs optimizer changes in 3rd release

This section describes changes in Jobs Optimizer in the third and final release of ANDREAS.

3.1 Jobs Optimizer API

The REST API to access the Jobs Optimizer has been modified as follows. As in the previous releases, the user interacts with the application through a POST request, whose details are presented hereunder.

Parameters:

Name	Description
body (required)	Set of jobs, node resources and parameters required for the optimization Model: {"jobs": {JobID (string): , {"SubmissionTime": "%a %d %b %Y, %H.%M.%S", "Deadline": "%a %d %b %Y, %H.%M.%S", "Priority": int, "Epochs": int, "ProfilingData": {GPUType (string): {nGPUs (string): time (float [s]), nGPUs (string): time (float [s]), ...}, GPUType (string): {nGPUs (string): time (float [s]), nGPUs (string): time (float [s]), ...}, ...}} }, "nodes": {NodeID (string): {"GPUtype": string, "free_nGPUs": int, "total_nGPUs": int, "cost": float [€/h]} }, "GPUcosts": {GPUtype (string): {nGPUs (string): cost (float [€/h]), nGPUs (string): cost (float [€/h]), ...}, GPUtype (string): {nGPUs (string): cost (float [€/h]), nGPUs (string): cost (float [€/h]), ...}, ...}, "currentScheduling": {JobID (string): {"expected_tardiness": float, [h] "nGPUs": int, "NodeID": string}, ...}, "currentTime": "%a %d %b %Y, %H.%M.%S", "method": string, "verbose": int } Parameter content type: json

	Note (1): all fields except currentScheduling, method and verbosity level are mandatory. **If provided, the currentScheduling will be used to minimize changes in the running configuration, reducing the time required to migrate jobs between different nodes.** **Note (2):** the Priority field is used by the optimizer to weigh the penalty due to deadline violations: jobs with higher priority will be charged with a higher penalty. Valid values are 1, 2, 3, 4 or 5, where 1 denotes a job with minimum priority, while 5 denotes a job with maximum priority. **Note (3):** the ProfilingData section stores information collected through profiling, concerning the execution times of each job with the available configurations, identified by a given type and number of GPUs. **Note (4):** Json format forces all keys to be provided as strings, therefore also the number of GPUs, when it is used as key, as in the ProfilingData and the GPUcosts sections, must be provided in this format and not as an integer. **Note (5):** for what concerns the unit of measures, the times to be provided in the ProfilingData section are expected to be in seconds. Vice versa, all costs should be provided in €/h. The expected tardiness provided in the currentScheduling section, which corresponds to the value returned by the optimizer in the previous call, should be provided in hours. **Note (6):** the cost provided in the nodes field denotes the cost related to a node when it is idle; vice versa, the costs provided in the GPUcosts field are the total costs incurred when the given number of GPUs is used. Details are provided in Section 4.2.

Responses:

The following error codes have been added to the ones listed in [1]:

Code	Output	Description
474	Body provided by the user	KEY ERROR Mandatory field "GPUcosts" missing in data
484	Body provided by the user	PARAMETER ERROR Missing data in the GPUcosts section

3.2 Code structure

The Jobs Optimizer has been implemented, as mentioned in the previous releases, in C++. In the last months, the code has been refactored and tested extensively. Moreover, a new component in charge to minimize the changes in the configuration of already running jobs, reducing the time required to migrate jobs between different nodes, has been implemented.

The access point is given by a POST request, based on Flask, whose characteristics are described in the previous section. The class diagram of the code is presented in Figure 3.

The components that are explicitly exploited by the ANDREAS optimizer are highlighted in red while the classes Simulator, Local_search and Path_relinking, as well as some derived classes of Heuristic, which will not be part of the ANDREAS code base and final product, are in light gray.

The data provided by the POST request, namely the list of submitted jobs and the characteristics of the available nodes and resources, are loaded in the class System, which is given as datum to the main class Optimizer. The goal of this class, through its method algorithm, is to build a set of good-quality solutions, denoted as Elite_solutions, by means of the Heuristic algorithm selected through the parameter method. The Solution with optimal cost is finally provided as output by the method.

The two available methods are EDF and Random_greedy. As described in [1] and [4], EDF (Earliest Deadline First) proceeds by sorting jobs according to their deadline, so that those that are most likely to violate it, receive resources as first. The available resources (in terms of nodes and GPUs) are partitioned among the jobs and each node can be shared by multiple applications. Once EDF allocates resources to a job, the job will continue to run without any change in the configuration until it will finish. The Random_greedy method, instead, is a more advanced heuristic algorithm that considers jobs in increasing order of pressure. This quantity measures how much jobs are close to their deadline when executed with the fastest configuration (type and number of GPUs), so that jobs that are more likely to violate the deadline (thus incurring a penalty cost) receive resources as first. Each job is assigned to the cheapest available configuration that allows it to be executed before the deadline (if this is possible), or the configuration that minimizes the violation. Nodes can be partitioned among multiple jobs to improve the resource usage. A detailed description of EDF and the random greedy method is available in the deliverable “D1 - 1st release of the integrated schedule” [1].

The final Solution returned by the Optimizer is provided as input to the method perform_analysis of the Analyzer class. This method, added in this last release, performs a comparison with the currentScheduling given as input to the POST request, if it is provided, and which describes the currently running configuration. The goal of this comparison, and the consequent adjustment, is to minimize the changes between the current and the next system configuration, in order to reduce the time required by jobs migrations.

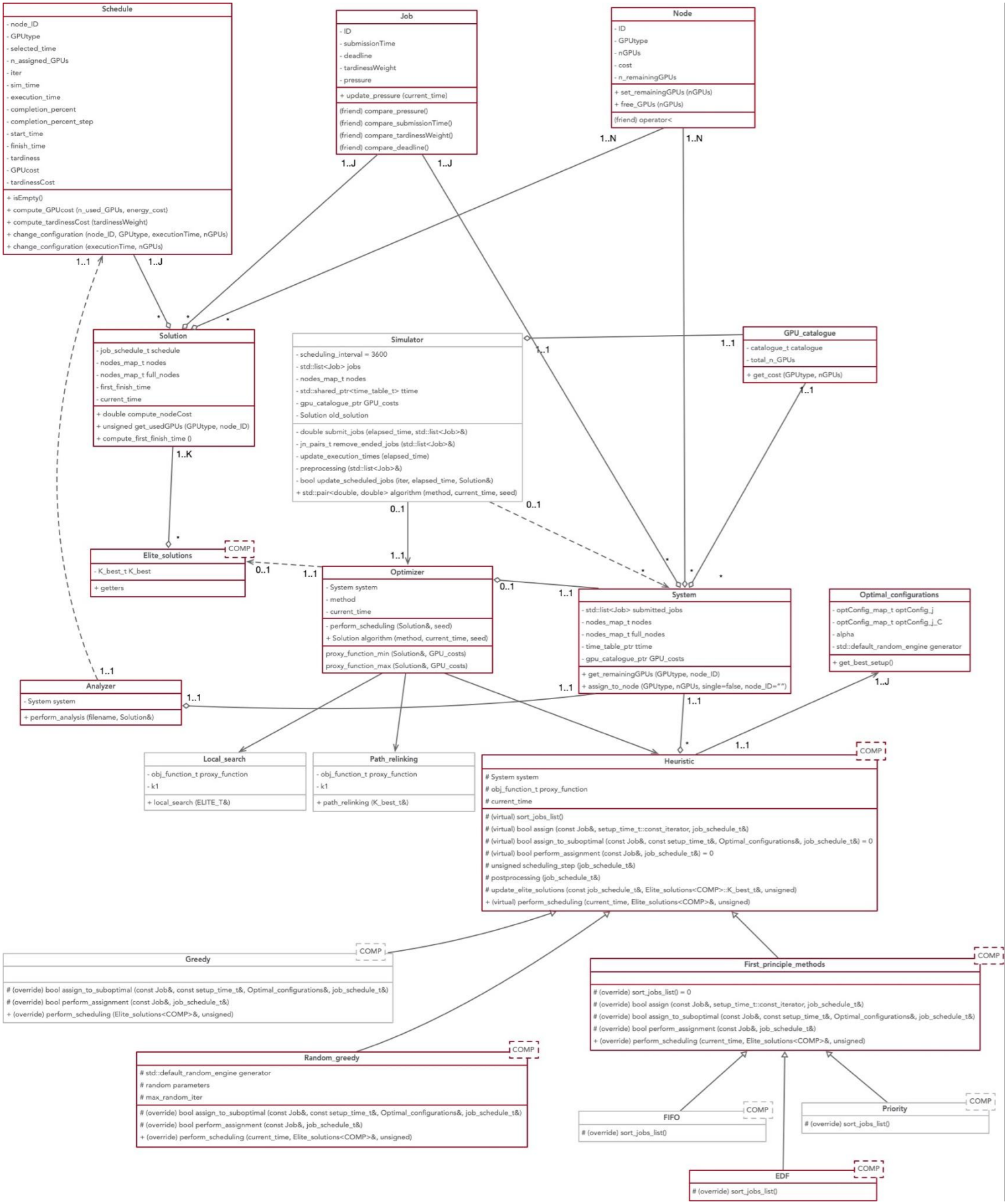


Figure 3. Jobs Optimizer Class Diagram

4. Testing results

This section describes the results obtained through an experimental campaign, aiming at assessing the effectiveness of ANDREAS framework in real-case scenarios. In Section 4.1 we describe the characteristics of the Deep Learning applications used for testing. In Section 4.2, we report the results of a profiling analysis, aiming at measuring the power consumption of ARMIDA nodes and the expected execution times of the considered jobs. In Section 4.3, we present the results of a set of simulations that compare the performance of the Randomized Greedy algorithm developed in the ANDREAS optimizer with those of first-principle methods, including EDF. Finally, in Section 4.4 we describe the results of the validation of our tool by exploiting the ARMIDA setup.

4.1 Applications understudy

In order to test the ANDREAS framework, we have considered heterogeneous Deep Learning jobs based on different network types, as described in Table 2. All the jobs are implemented relying on TensorFlow 2. LSTM jobs predict values of EUR/USD exchange rate based on historical data. Multiple layer LSTM networks are used as a model. LSTM-big has a learning rate of 0.00001 while LSTM-small of 0.000001. EfficientNet script uses Keras built-in EfficientNetB0 network to create a model to recognise images from enlarged CIFAR-10 dataset (10 classes). ConvolutionNet uses a sequential model of convolution layers to recognize numbers from an enlarged MNIST image dataset.

Application	**Dataset Size**	**Profiling Epochs**	**Batch size**
LSTM-big	16788 records	10	8192
LSTM-small	16788 records	10	8192
efficientNet	480k images	10	4096
convolutionNet	1920k images	10	8192

Table 2. DL jobs characteristics

The batch size of the aforementioned jobs has been selected with the goal of maximizing GPUs usage.

4.2 Profiling results

The profiling analysis described in this section has been performed to measure the power consumption of ARMIDA nodes and GPUs, as well as the expected execution times of the applications described in Section 4.1 on all the available configurations. The reference hardware system includes four nodes configured as in Table 3.

Node ID	GPU type	Number of GPUs
armida-04	TESLA	1
armida-05	TESLA	1
armida-06	TESLA	2
armida-07	TURING	1

Table 3. Characteristics of the target nodes

The Jobs Manager allows to specify, first of all, the node that should be used for profiling, that can be changed over time to retrieve data for different types (and, possibly, numbers) of GPUs (note that the GPU types within each node are homogeneous). Once the profiling node is fixed, every time a new job is submitted, it is executed for a given number of epochs, collecting the corresponding execution time. Each job is characterized by an ID, which is related to its type. The first time the job is submitted, it is profiled on one GPU. Then, the number of GPUs is increased every time an analogous job with the same ID is submitted, in order to increase the catalogue of execution times available for that type of job.

The expected execution times need to be provided to the Optimizer, since a job cannot be assigned to a given configuration (type and number of GPUs) if the expected execution time on that configuration is not available. They have been obtained by performing a 10 epoch profiling and are reported in Table 4.

Application	GPU type	GPU number	Expected execution time per epoch [s]
LSTM-big	TESLA	1	43.0842
LSTM-big	TESLA	2	30.5512
LSTM-big	TURING	1	110.7112
LSTM-small	TESLA	1	43.0261
LSTM-small	TESLA	2	30.5871
LSTM-small	TURING	1	110.6581
efficientNet	TESLA	1	51.6221
efficientNet	TESLA	2	41.2372
efficientNet	TURING	1	88.9881
convolutionNet	TESLA	1	20.5382
convolutionNet	TESLA	2	17.8042
convolutionNet	TURING	1	41.2582

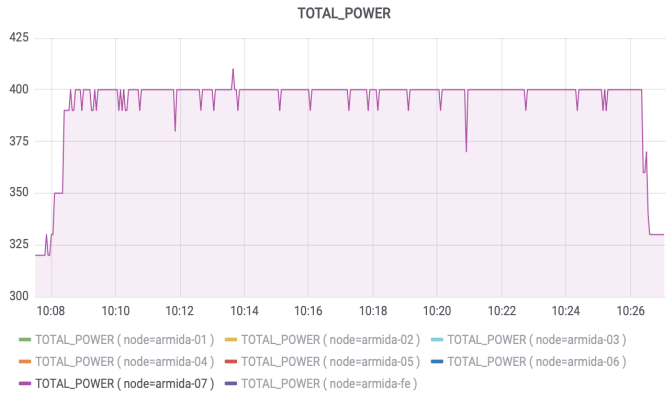
Table 4. Profiling results

As previously mentioned, the profiling campaign has been devoted also to measure the power consumption of the different nodes available in the Armida system, in order to compute the hourly energy costs of job executions. The results of this process are shown in Figure 4.

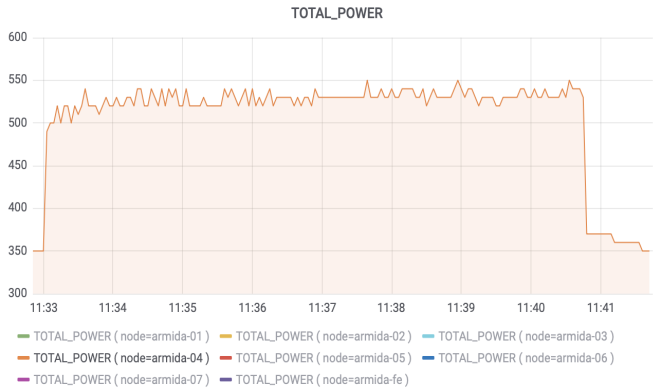
The power consumption of ARMIDA nodes, and, in particular, of the different type and number of GPUs, is used to compute the hourly energy costs used by the optimizer to determine the best assignment for the different jobs.

To evaluate also the energy consumption due to the air conditioning system, the UPS and other overheads of a datacenter environment, the energy costs has been inflated by considering a PUE (Power Usage Effectiveness) [8] equal to 1.33, which is the value measured in the ARMIDA setup and reference for the data centers of the potential customers of the product.

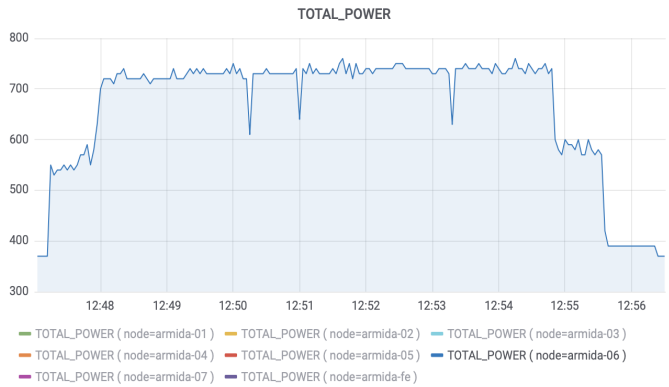
The power consumption measured as in Figure 4, the hourly energy cost and the PUE are used *a priori* to determine the final values given to the optimizer, as described in Sections 3.1 and 5.1.



GPU type: TURING - number of GPUs: 1



GPU type: TESLA - number of GPUs: 1



GPU type: TESLA - number of GPUs: 2

Figure 4. Power consumption of ARMIDA nodes

4.3 Exploratory analysis

The results that can be achieved by exploiting the ANDREAS optimizer at a larger scale have been evaluated through a set of simulations, comparing the outcome of the Randomized Greedy (RG) algorithm used by the Optimizer against first-principle methods as First In First Out (FIFO), Earliest Deadline First (EDF) and Priority Scheduling (PS).

The jobs considered for the analysis are the ones described in Section 4.1. We have generated multiple scenarios, considering a number N of nodes varying from 10 to 100, and a number of jobs J = 10N. Inter-arrivals of jobs are characterized by a *mixed* rate as described in [1]. The expected execution times of the different jobs with the available configurations have been retrieved through the profiling step described in the previous section.

In each instance, the system we have considered is characterized by a fixed number of nodes. Half of them are equipped with GPUs of type TESLA, while the others with GPUs of type TURING. We have considered two different cases: in the first one, nodes are characterized by 2 GPUs of type TESLA or by 1 GPU of type TURING, whose expected energy costs have been measured as described in Section 4.2. In the second case, we have used the costs data measured in the previous section to extrapolate the expected costs of larger

systems, thus we have considered nodes with 4 GPUs of type TESLA or 2 GPUs of type TURING.

The results of the Randomized Greedy algorithm and of FIFO, EDF and Priority Scheduling methods in the first scenario are reported in Figure 5.

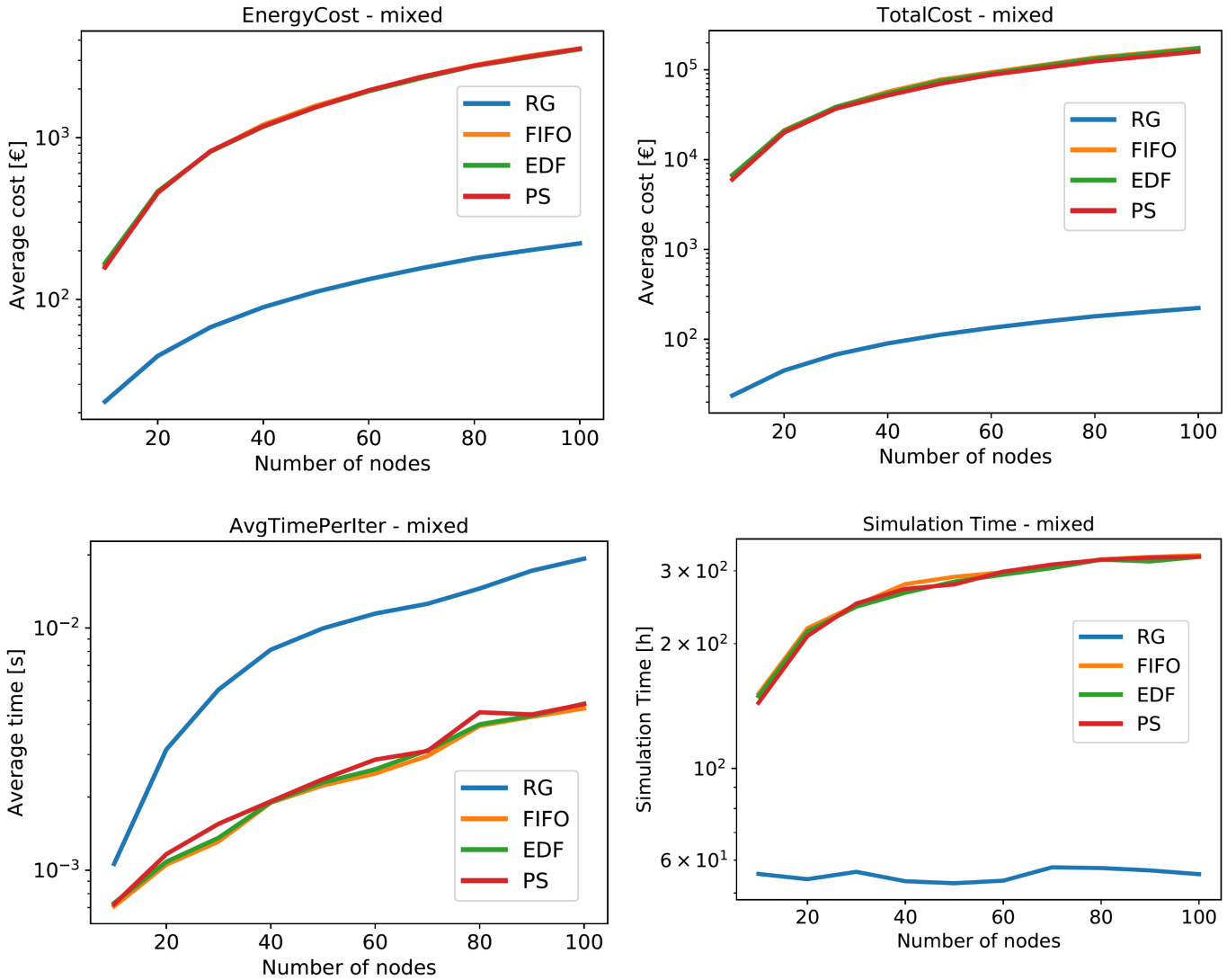


Figure 5. Jobs Optimizer results - first scenario

The first plot reports the expected energy costs of jobs execution, while the second shows the total cost, which is the sum of the energy cost plus the penalties due to jobs' deadlines violations. The Randomized Greedy algorithm guarantees the reduction of energy costs by one order of magnitude on average, with an average percentage gain over EDF method between 86% and 92%. An even more significant reduction is obtained considering the total costs. Indeed, first-principle methods never change the configuration assigned to a job once it has been started, which entails that, if jobs with higher priority are submitted after long-running jobs with lower priority, they have to wait for them to be completed before receiving resources.

The third plot reports the time required to the optimizer to solve one instance of the problem, answering the call of the Jobs Manager providing the schedule of all jobs. Even if our Randomized Greedy algorithm requires, on average, 3.7 times more than first-principle

methods to provide a solution, this time remains always below 20 milliseconds, which is significantly lower than the time required to reconfigure the system.

The last plot shows the expected time required to completely execute the training of all the J jobs submitted to the system. It is possible to notice that the Randomized Greedy algorithm reduces it by 5 times on average, thus minimizing the probability that jobs deadlines are violated and consequently reducing the corresponding penalty costs.

Figure 6 reports the comparison between the performance of the Randomized Greedy algorithm and the first-principle methods in the second scenario, involving nodes with either 4 GPUs of type TESLA or 2 GPUs of type TURING. The outcomes are similar to the ones presented above, which enhances the quality and effectiveness of our algorithm in tackling real-life scenarios.

The energy costs reduction guaranteed by our Randomized Greedy algorithm over EDF method is between 89% and 94%, while the expected time to completely execute all jobs is reduced by 3 times on average.

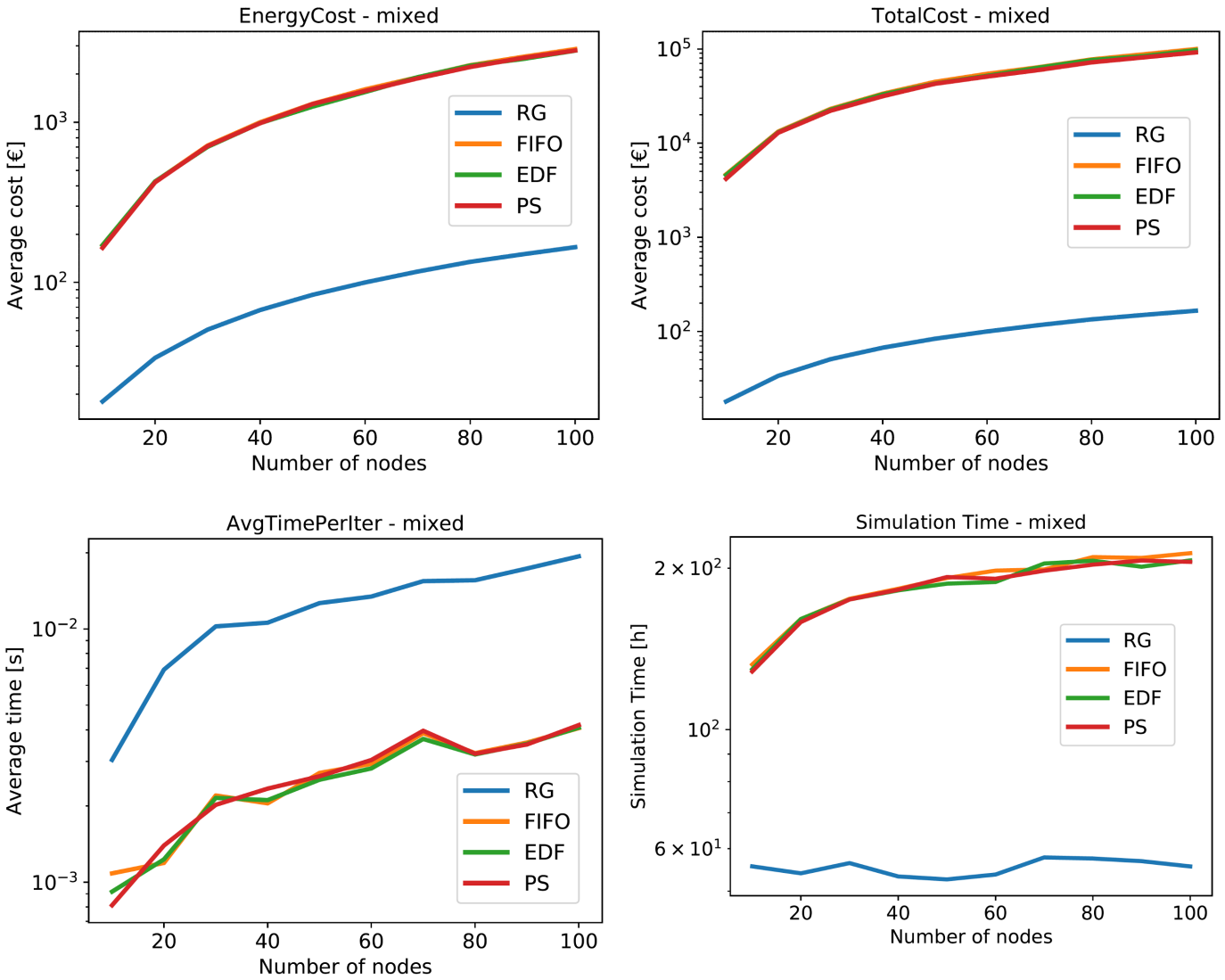


Figure 6. Jobs Optimizer results - second scenario

4.4 Validation in the ARMIDA setup

The results provided by ANDREAS Optimizer have been evaluated in the ARMIDA setup by considering a prototype system including 8 different jobs and exploiting heterogeneous ARMIDA nodes with different types and numbers of GPUs.

The characteristics of the submitted jobs are summarized in Table 5. The inter-arrival time has been set to 1200s. The considered scenario has been accelerated, in terms of submission frequency and average execution times of applications. Nonetheless, this does not affect the results effectiveness since the workload assigned to the available nodes is comparable to practical situations. The system is composed by three nodes, which correspond to armida-05, armida-06 and armida-07 as described in Table 3; the node armida-04 is not considered, since the Jobs Manager implementation requires that one among the available nodes is dedicated to profiling.

Job	Application	Epochs	Batch size	Submission Time [s]	Deadline [s]	Priority
J6	efficientNet	80	4096	0	3600	4
J9	convolutionNet	160	8192	1200	2600	2
J10	convolutionNet	80	8192	2400	7600	3
J7	LSTM-big	160	8192	3600	17600	3
J8	LSTM-small	160	8192	4800	7600	3
J1	LSTM-big	60	8192	6000	5600	5
J2	LSTM-small	60	8192	7200	12600	2
J3	efficientNet	60	4096	8400	11600	1

Table 5. Applications submitted to the system

The aim of this experiment was to assess the accuracy of our framework, by measuring the gap between the predicted energy costs and the actual costs measured in the ARMIDA nodes.

The sequence of jobs executions, with the relative configurations, is reported in Figure 7. Due to the fact that most of the jobs have strict deadlines, the two GPUs available on node armida-06 are never shared among different applications. Indeed, as it can be noticed in the results of profiling (Table 4), using two GPUs of type TESLA guarantees the lowest execution time, which results in a lower probability of violating the deadline. For the same reasons, the two GPUs are immediately assigned to J6, which maintains this configuration until J9 is submitted. This second job, even if it has a lower priority, has a significantly more stringent deadline, so that it is assigned to armida-06 in order to reduce the expected tardiness. J10, instead, is assigned to armida-05 and it is completely executed without violating the deadline. The execution of J7, which is the job with the farthest deadline, is stopped twice. This preemption allows freeing resources that are used to execute jobs with

higher priority or, in the considered case, more stringent deadlines. In particular, when J2 is submitted, resources are not enough to concurrently execute the four jobs. J1 and J8, which have higher priority than J10 and closer deadlines, cannot be preempted, to avoid the risk of incurring into tardiness penalties.

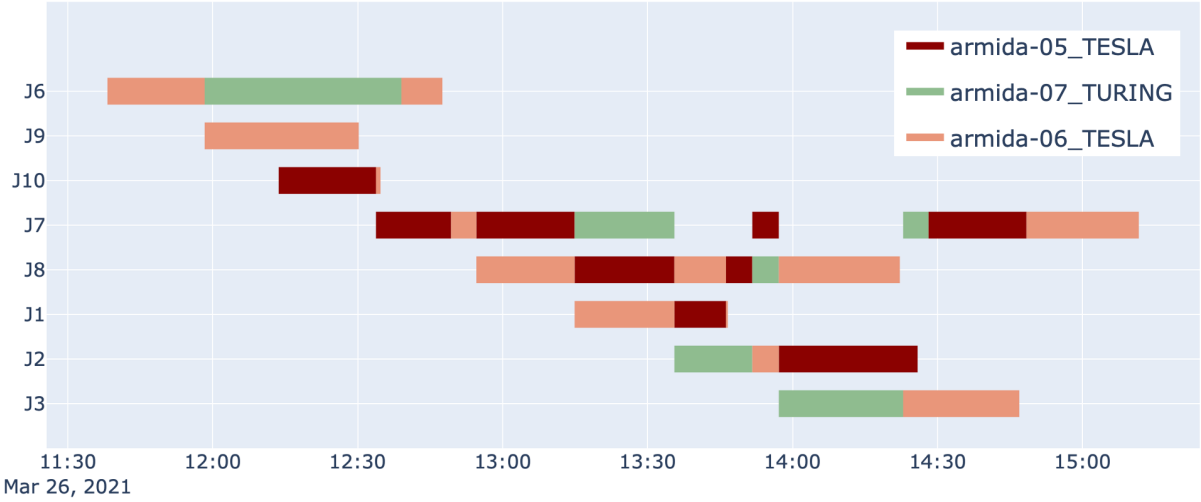


Figure 7. Results on the target system under Randomized Greedy Scheduler

Figure 8 reports the power consumption of ARMIDA nodes during the full experiment session. This has been used to compute the actual energy costs incurred by the system during the whole experiment, including the reconfiguration costs that are not considered by our simulator. We have compared this result with our prediction, considering three different aspects: first of all, we have compared the energy costs predicted by the simulator with the true energy costs. Then, we have compared the costs due to deadline violations, and finally the total costs given by the sum of these two terms. Table 6 reports the deviations of the predicted costs with respect to the real costs.

	Energy Cost	**Tardiness Cost**	**Total Cost**
Real Cost [€]	0.96028	3.03593	3.99621
Predicted Cost [€]	1.04364	3.12345	4.16710
Deviation	7.98801%	2.80215%	4.10094%
Average deviation for each optimizer call	6.63064%	4.14744%	5.36208%

Table 6. Results of the validation tests under Randomized Greedy Scheduler

It is relevant to notice that the predicted costs are always higher than the real costs, for all the terms we have considered. This, together with the fact that the measured deviations are always lower than 8%, proves the accuracy of our framework in estimating the execution costs of the considered applications and validates its adoption in an industry setting.

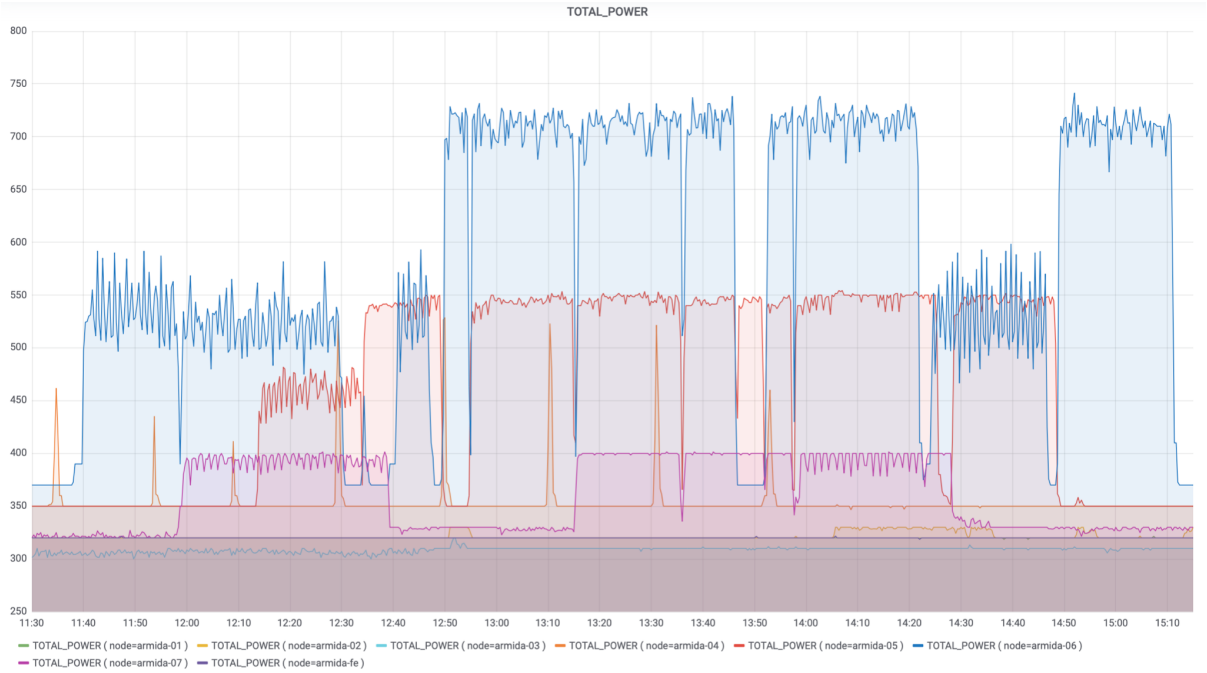


Figure 8. Power consumption of ARMIDA nodes under Randomized Greedy Scheduler

As an additional test, we have compared the real costs on the ARMIDA system when exploiting our Randomized Greedy Scheduler with those measured when exploiting an Earliest Deadline First (EDF) Scheduler. As mentioned in the previous sections, the EDF Scheduler does not allow jobs configurations to be changed after they have been assigned. Therefore, jobs are always executed completely on the resources they receive. Moreover, jobs cannot be preempted, which entails that, if a job with high priority is submitted after a long-running job with low priority, and resources are not enough to run both concurrently, the high-priority job will likely incur in a deadline violation.

The results of the comparison between the Randomized Greedy and the EDF Scheduler are summarized in Table 7. The simplified scenario we have considered does not allow to see a significant difference in terms of energy costs when comparing the two methods, while the difference is of around 30% when considering also the penalty costs due to deadline violations.

	Energy Cost	**Tardiness Cost**	**Total Cost**
Real Cost Randomized Greedy [€]	0.96028	3.03593	3.99621
Real Cost EDF [€]	0.96448	4.85930	5.82378
Deviation	0.43559%	37.52331%	31.38118%

Table 7. Results of the validation tests under EDF Scheduler

The sequence of jobs executions, with the relative configurations, under the EDF Scheduler is reported in Figure 9, while Figure 10 reports the power consumption of the ARMIDA nodes in the considered scenario.

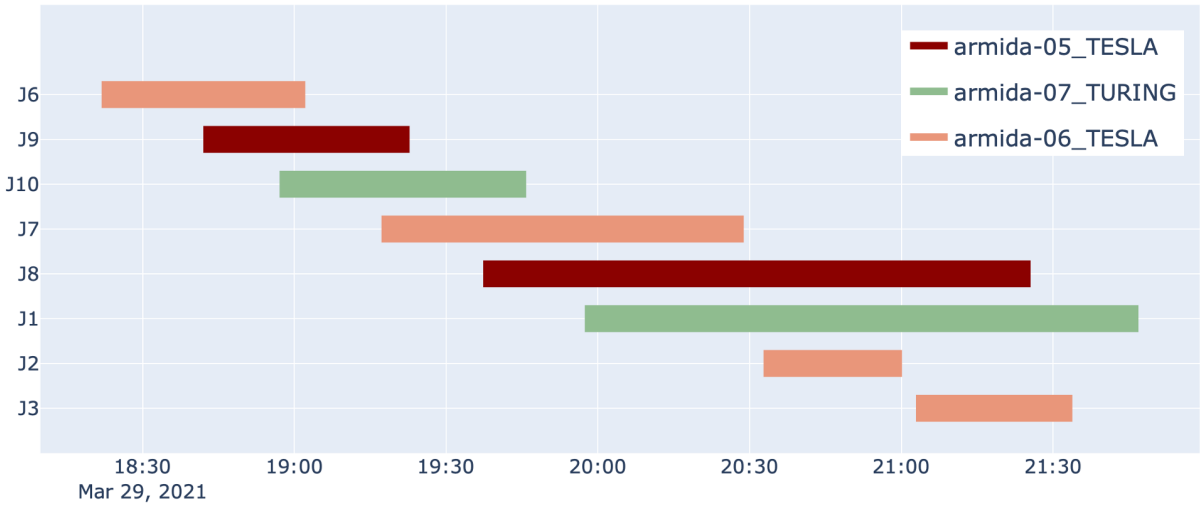


Figure 9. Results on the target system under EDF Scheduler

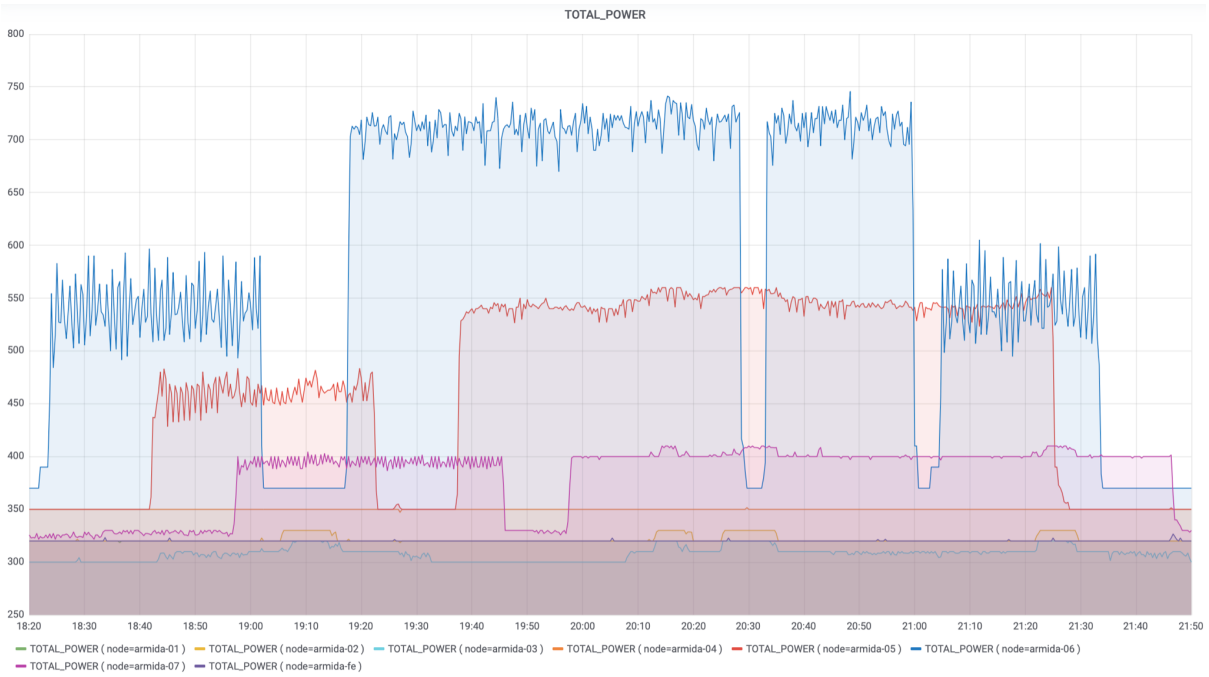


Figure 10. Power consumption of ARMIDA nodes under EDF Scheduler

5. User guide

This section describes how to prepare the jobs and use the ANDREAS framework.
The ANDREAS project code base is provided as a folder called “Andreas”. Inside, there are two subdirectories: “startApplication” (containing all files needed to start the Jobs Manager) and “useApplication” (example of usage of the framework, assuming the Jobs Manager is already running). Section 5.1 illustrates how to set up the configuration file (placed in the “startApplication” folder) of the Jobs Manager where one could change properties of the application and GPU energy costs specific for the platform it uses. Sections 5.2 and 5.3 guide on how to prepare a working directory as well as the scripts of the Machine Learning jobs that the end user wants to launch. Examples of working directories are placed in the “useApplication/testingExample” folder. Note that, from now on we will be using interchangeable notions “user’s script” and “job’s script” to describe the Machine Learning script prepared by the end user.
Section 5.4, finally, explains how to start the ANDREAS program as a java process: this step requires that the Optimizer and database containers have been previously launched. This section is followed by a specific description of how to use ANDREAS and how to submit user’s jobs (prepared earlier according to the discussion in sections 5.2 and 5.3).

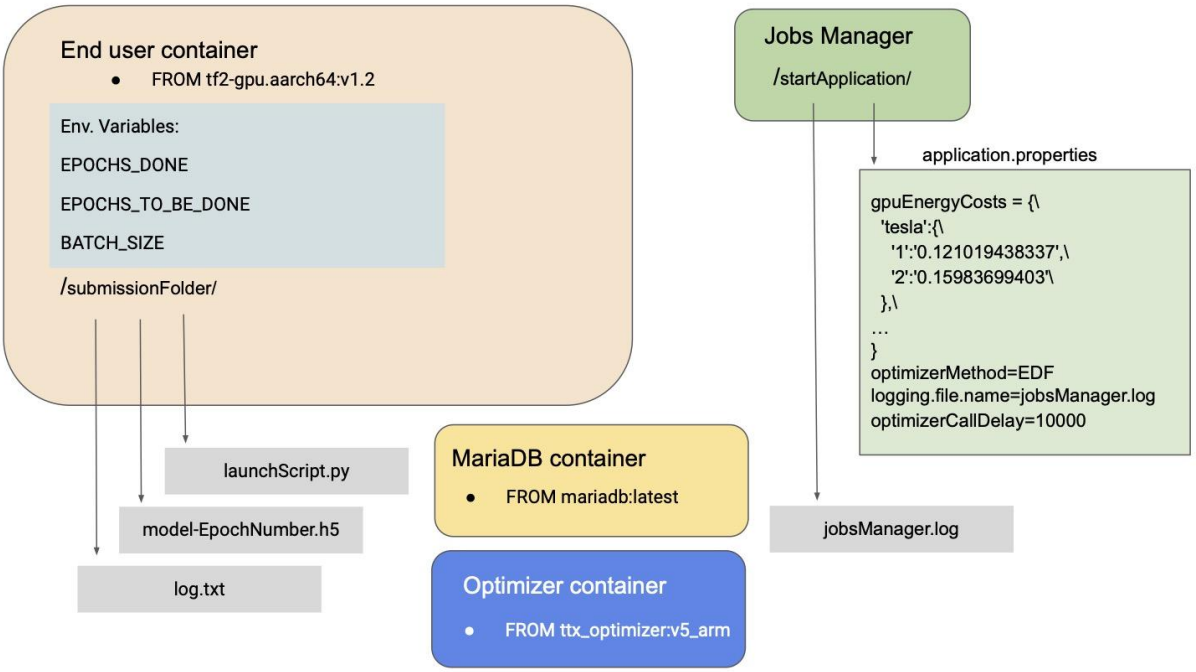


Figure 11. Architecture of the ANDREAS deployment on ARMIDA.

5.1 Jobs Manager configuration file

Inside the “startApplication” subdirectory there are all files needed for the proper work of the entire application (java executable file, Jobs Manager launching scripts). The configuration file “application.properties” of the jobsManager is also included (see Figure 11). In this file, one can change the values of power usage consumption that will be passed to the optimizer.

Attribute GPUcosts in the JSON message (described in the section 3.1) would be populated according to the map from this properties file:

```
gpuEnergyCosts = {\
  'tesla':{\
    '1':'0.121019438337',\
    '2':'0.15983699403'\
  },\
  'turing':{\
    '1':'0.09133542516'\
  }\
 }
```

Inside this file one could also change other properties of the Jobs Manager, such as the level of logging or name of the file that will contain the logs from the Jobs Manager (by default this file will appear inside startApplication directory). The time interval between two subsequent calls to the optimizer (in milliseconds) can also be changed. User can also choose between two methods used by the Optimizer for finding optimal solution (i.e., Random Greedy and EDF) as follows:

```
logging.file.name=jobsManager.log
optimizerCallDelay=10000
#EDF (Earliest Deadline First) or RG (Randomized Greedy)
method=EDF
```

5.2 How to prepare setup folders

Each job instance needs to have a setup directory, dedicated for this instance only. Those can be any directories visible from the nodes and accessible for the user (a simple “mkdir directoryName” would be sufficient to create such a folder). All input files should be included inside. The files inside this directory cannot contain numbers in their names. This directory will be mounted to the docker container during the training process. Its name would need to be specified during submission of the job. Examples of working directories hierarchy can be found in the “Andreas/useApplication/testingExample” folder.

5.3 How to prepare the scripts to be launched

Each user’s job will be launched inside a docker safe environment. Inside this docker container, the user’s job script (its absolute path needs to be defined in the submition’s Json message: see section 5.5) will be run with “python3” command. It is important that the script uses only relative paths, assuming that it will be launched inside the working directory containing all needed input files (see section 5.2). The output files will be saved inside this directory(“./” relative path should be then used inside the user’s script for any output files).

To use all the features of the application, the job’s script needs to save the trained model after each epoch. The name of this file needs to contain the number of epochs already computed (for example a correct name for a model file name is “model-1.h5”). No other numbers are allowed to be used in the names of the files (both input and output), except for the file extensions.

The script will be run inside the docker container defined during the job's submission. The docker container should contain the environment that fully enables the launching of the script, i.e., all the required python libraries need to be preinstalled. Inside the docker the Jobs Manager would export three environmental variables (see Figure 11):

- 'EPOCHS_DONE" - value indicating how many epochs were already processed. In the example scripts, it provides the information on which snapshot needs to be loaded when a job is relaunched.
- 'EPOCHS_TO_BE_DONE' - value indicating how many epochs still need to be processed.
- 'BATCH_SIZE' - batch size defined during submission of the job.

The most important parts that should be included in the job's script (launchScript.py in Figure 11) are the following (each point is followed with the example implementation that were used in our testing scripts):

1. Enabling gpu usage:

```
config = tf.ConfigProto()

config.gpu_options.allow_growth = True

tf.keras.backend.set_session(tf.Session(config=config))
```

2. Loading snapshot based on environmental variable:

```
epochs_already_done = int(os.environ.get('EPOCHS_DONE'))

model_path_to_load = 'model-'+ str(epochs_already_done)

path = model_path_to_load + '.h5'

model = tf.keras.models.load_model(path)
```

3. Using distributed strategy to improve the training with multiple GPUs:

```
strategy = tf.distribute.MirroredStrategy()

with strategy.scope():

  #here is the code preparing model and launching training
```

4. Saving model after each epoch:

```
for i in range(1, epochs_to_be_done+1):

   model.fit(x_train, y_train, epochs=1, batch_size=batch_size, shuffle=True,

                validation_data=(x_test, y_test))

   model_path_to_save = 'model-'+ str(epochs_already_done + i)

   model.save(model_path_to_save+'.h5')
```

Note that in our testing example scripts, we also provide simple logging and after each epoch, the loss value is saved to log.txt (inside the working directory, see Figure 11).

5.4 How to start the application

As a first step, load the following docker images into the docker service on the control node of the platform. On ARMIDA those images are already uploaded:

- ttx_optimizer:v5_arm
- mariadb:latest

As a second step, run the application from the “Andreas” folder by calling the script: ./startNew.sh.

This script would delete and restart other components (the Optimizer and the database containers) and shut down running containers of the jobs that have already been submitted. A database restart will also clean all the previous profiling data. If you want to start only the Jobs Manager (using an already running database and the Optimizer), run: ./startAndContinue.sh script. In this way, the JobsManager would continue the former workflow (if, for example, it was stopped earlier by “ctrl + c” signal).

5.5 How to manage running applications

Reaching the REST API of the Jobs Manager can be done in two ways. The first REST call, setup the node environment, while the second allows jobs submission.

5.5.1 Sending JSON Messages

Postman program enables to send JSON from an external device. Example Postman collection (Json requests of the example testing workflow) can be found in the “Andreas/useApplication” directory (and can be imported into the program). Alternatively one can create the file containing the Json text and launch:

curl -X POST -H "Content-Type: application/json" -d @req.json http://localhost:8080/nodes

changing endpoint details if necessary. “req.json” is the name of the Json file mentioned above.

5.5.2 Setting up the system configuration

As a first step, there is the need to inform the Jobs Manager about the nodes in the system, using a JSON file with a specification of which nodes will be dedicated for the profiling. The system itself will recognize the available GPUs attached to them.

The message is a JSON file sent to the localhost:8080/nodes endpoint. An example of setup used in ARMIDA is the following:

```
[
  {
    "id" : "armida-04",
```

"isForProfiling" : false,
"cost" : 0.079918497015
},
{
"id" : "armida-05",
"isForProfiling" : false,
"cost" : 0.079918497015
},
{
"id" : "armida-06",
"isForProfiling" : true,
"cost" : 0.079918497015
},
{
"id" : "armida-07",
"isForProfiling" : false,
"cost" : 0.073068340128
}
]

- “Id” : this is the name of the node available in the system and connected to the control unit (armida-fe on ARMIDA).
- “isForProfiling”: set to true determines that the node is dedicated for the profiling.
- “Cost”: this value is passed forward to the optimizer to find the optimal jobs assignment. This field corresponds to the attribute “cost” of “nodes” section in the JSON sent to the Optimizer and described in the section 3.1, thus it corresponds to the cost related to the different nodes when they are idle.

Note that this message can be sent multiple times, changing the setup of nodes (for example changing nodes dedicated for profiling, or adding or deleting nodes) anytime is needed. Current jobs, possibly, would be then be moved to the available nodes, according to the new Optimizer’s assignment obtained at the next optimization call.

5.5.3 Job submission

As a second step, jobs can be submitted to the system, again using a JSON message. All paths and other properties should be specified inside this message.

Jobs submission should be sent to the localhost:8080/submit endpoint. An example is reported below:

```
{
  "job_id" : "LSTM-small",
  "dockerImage": "tf2-gpu.aarch64:v1.2",
  "scriptPath": "/home/user/scripts/snapLstmOneStepSmall.py",
  "directoryToMount": "/home/user/lstmJobs/job_lstm_3",
  "Priority": 1,
  "deadline": "2021-02-28T17:09:42.411",
  "batchSize" : 2048,
  "profilingEpochsNo" : 2,
  "epochsTotal": 15
}
```

A summary of the fields semantic is the following:

- "job_id" - It is the unique ID identifying the job. If you launch the same script multiple times you should put here the same name as it allows to create additional profiling data and leads to a better job assignment
- "dockerImage" - we highly encourage to use the container image developed within the ANDREAS project as base image as it contains everything that is necessary to launch the script on ARMIDA with GPU support
- "scriptPath" - path to the script. Many jobs can use the same script file
- "directoryToMount" - this is the job working directory required for each of the jobs, as described in the section 5.2. Different directories must be used for different submissions.
- “Priority”: priority of the job. This is an integer ranging from 1 (minimum priority) to 5 (maximum priority) and it is used by the Optimizer to determine the order of jobs execution. Further information are provided in the JSON description in Section 3.1
- "deadline": deadline of the job. Specific format needs to be used as in the example above
- "batchSize": the batch size that will be exported into the docker container as an environmental variable
- "profilingEpochsNo" - number of epochs that will be used during profiling. More epochs give more accurate estimates for the job. However, this value should be lower than the total number of epochs that need to be processed
- "epochsTotal": - total number of epochs for the job

6. Business Plan

The Business Plan for ANDREAS has been developed considering the current business environment:

- Market impacted by the ongoing COVID-19 pandemics
- The need of the end-users for maximizing the resources (primarily GPUs) via effective and efficient solutions
- The growing market of ML solutions
- The option provided to the end-users to test the product in 'as-a-service' mode at an attractive price point before deciding whether purchasing the product (via a licence fee) or continuing with the 'as-a-service' mode (via an as-a-Service licence)

The Business Plan as of March 2021 is reported in Appendix A.

7. Conclusion

This document describes the final activities performed within the ANDREAS project and summarizes the changes compared to the first and second releases of the product.
In the final stage of the project, we have focused on improving quality towards commercial use of the platform and further improving the ANDREAS scheduler to include feedback after final release testing and evaluation from the early adopters. Simulations have shown how an average reduction in energy costs between 86 and 94% can be achieved when exploiting our algorithm with respect to Earliest Deadline First and a speedup of the applications executions between 3x and 5x, on average. Also, the extensive testing on on-premise environment has been performed as described in Section 4. The comparison between the predicted and the real costs on the ARMIDA system shows deviations lower than 8%, proving the accuracy of our framework in estimating the energy consumption costs and its suitability for its adoption in an industry setting. Finally, a demo account andreas_test has been setup as a landing page describing the product.
The objectives of the project as per the proposal submitted to the TETRAMAX-VALUECHAIN-TTX-3 Horizon 2020 innovation action have been achieved, and the product is ready to hit the market. The Business Plan and Partners' Agreement are in place and are currently executed. The ANDREAS team is committed to take the appropriate actions to make ANDREAS a technical advancement, a commercial success and an opportunity for the professional growth of the staff.

8. References

1. D. Ardagna et al. “D1 - 1st release of the integrated schedule”, 2020.
2. F. Beneventi, A. Bartolini, C. Cavazzoni, L. Benini, “Continuous learning of HPC infrastructure models using big data analytics and in-memory processing tools”, Design, Automation & Test in Europe Conference & Exhibition (DATE), 2017 https://ieeexplore.ieee.org/abstract/document/7927143
3. A. Bartolin, F. Beneventi, A. Borghesi, D. Cesarini, A. Libri, L. Benini, C Cavazzoni, “Paving the Way Toward Energy-Aware and Automated Datacentre”, ICPP 2019: 48th International Conference on Parallel Processing, https://dl.acm.org/doi/abs/10.1145/3339186.3339215
4. P.Skrzypek et al.“D2 - Testing and 2nd release of the integrated scheduler”,2020.
5. Hochreiter, Sepp. "The vanishing gradient problem during learning recurrent neural nets and problem solutions." International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems 6.02 (1998): 107-116.
6. LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. Proceedings of the IEEE, 86(11), 2278-2324.
7. Tan, M., & Le, Q. (2019, May). Efficientnet: Rethinking model scaling for convolutional neural networks. In International Conference on Machine Learning (pp. 6105-6114). PMLR.
8. Google. Efficiency in Data Centers. https://www.google.com/about/datacenters/efficiency/

Appendix A/Business Plan

ANDREAS

Artificial intelligence traiNing scheDuler
foR
accElerAted clusterS

A project funded within the
TETRAMAX/Technology Transfer via
Multinational Application Experiments

Business Plan

Fabrizio Magugliani/E4 Computer Engineering SpA
Konrad Wawruch/7bulls
Created on February 17th, 2021
Updated on March 19th, 2021
Updated version on March 22nd, 2021
Final version on March 30th, 2021

Executive Summary

Product

ANDREAS is the outcome of the project ANDREAS, funded within the TETRAMAX/Technology Transfer via Multinational Application Experiments and executed between June 1st, 2019 and Feb 28th, 2021 by Politecnico di Milano (IT), 7bulls.com (Poland) and E4 Computer Engineering SpA (IT). The TETRAMAX project has received funding from the European Union's Horizon 2020 research and innovation programme under grant agreement number 761349.

The objectives of ANDREAS are:

- Provide cloud providers and users a product targeted at minimising the resource energy footprint for AI workloads by maximising resource usage efficiency
- Develop a product featuring advanced scheduling solutions optimizing Deep Learning applications training run-time performance and their energy consumption in accelerated GPGPU clusters.

Customers

The target audience for ANDREAS are cloud providers and Compute/Data centers providing services to users for performing training of networks in the field of Machine Learning.

Future of the Product

Our goal is to build and market a tool that will be marketed and supported beyond the duration of the project. While the funding has been used by the partners to develop and test the product, the actual selling process, as well as the support of the products, will be performed by the partners as self-funded activities, enabling through the revenue stream of the sales the support of the products and further development according to the requests of the users.

Partners' Description

E4 Computer Engineering

Since 2002, E4 has been innovating and actively encouraging the adoption of new computing and storage technologies. With a comprehensive range of hardware, software and services, we are able to offer you solutions for the most demanding workloads: HPC, Big-Data, AI, Deep Learning, Data Analytics, Cognitive Computing and much more. We take an agnostic approach towards the many technologies available on the market. This allows us to devise and offer the best solution tailored to customer needs.

The Mission

Getting the best from innovation and making it easy. Implementing and integrating the most advanced technologies to obtain the best performance from computing and storage. Producing high quality solutions starting from requirements analysis to after-sales services to provide a full customer support.

The Vision

Becoming the reference supplier for IT solutions, focused on technological innovation, with the aim to make our customers develop and implement their own research and improve the world of tomorrow.

Politecnico di Milano

Politecnico di Milano is one of the largest Italian universities ranked as one of the top in EU and 16th worldwide in the Engineering & Technology category (2019 QS World University Rankings).
Politecnico di Milano was established in 1863 and is one of the largest Italian universities, with 5 campuses, 9 Schools (6 of Engineering, 2 of Architecture, 1 of Industrial Design) and over 40.000 students. Research is carried out on a wide range of technical and scientific subjects by 16 Departments. POLIMI is ranked as one of the most outstanding European universities: according to the 2019 QS World University Rankings, Politecnico di Milano is the 16th university in the world in the Engineering & Technology category, 11th in Architecture and 6th in Design.
Politecnico participates in this project through members of Dipartimento di Elettronica, Informazione e Bioingegneria (Italian for Department of Electronics, Information and Bioengineering) with the group led by Prof. Danilo Ardagna (http://ardagna.faculty.polimi.it). The research activity focuses on techniques, tools, and frameworks for the development and runtime management of complex software systems. It encompasses a variety of aspects of such systems, ranging from modelling and analysis issues in the early phases of their development to issues related to their implementation and runtime management, self-adaptation, and optimisation.

7bulls.com

We are a Polish private company developing software in the newest technologies. We've been dynamically developing for over 15 years. We provide implementation and integration IT services and create comprehensive IT solutions. We work for companies from North and South America, Asia, Western Europe and Poland.
We deliver new, advanced technologies for large organizations, including financial, publishing, automotive, and pharmaceutical companies, as well as media corporations and sales chains.
We have the status of a Research and Development Centre in Poland and in France.

Market Research

Machine Learning

AI is becoming pervasive for Western Europe and is projected to yield a buoyant 49.9% CAGR through 2022 reaching USD 1.6 billion in revenues (https://www.idc.com/getdoc.jsp?containerId=EMEA42110117) while AI investments at EU level are expected to reach a total (public and private sectors) of at least EUR 20 billion between 2018-2020, with about EUR 20 billion increase per year in the next decade (see EC Coordinated Plan on AI).
Although the demand for GPU-powered clusters for running AI models is skyrocketing, the deployment of large/medium GPU-powered clusters is restrained by power consumption and cooling. A fully functional system for industrially significant AI-related workloads (e.g, the NVIDIA DGX STATION) consumes about 1500 W, while for top-of-the line systems (e.g., NVIDIA HGX-2) almost 8 times more.
A preliminary analysis performed by E4 suggested that in industry clusters GPUs utilization ranges in 25-50% but without the corresponding energy saving wrt peak power consumption.
This industry is currently suffering from the economic recession as consumers are spending less on non-essentials. However, industry revenues increased by $1.2 million in the 2nd quarter of 2012. This means there is a potential for growth as the economy recovers.
Based on estimates from Hyperion Research, IDC, and E4 internal market researchers, the market for GPU-powered servers had a worldwide total addressable market of $37 billion in 2018. Looking ahead to 2023, the total addressable market for the GPU-powered servers for technical/scientific application is expected to grow by around 35% to $20 billion, about 40% of the $50 billion total addressable market for servers.

Detailed Description of Customers

Cloud providers: Cloud providers (or, cloud service providers) are companies that provide IT environments like public clouds or managed private clouds that abstract, pool, and share scalable resources across a network. Cloud providers can also offer components of cloud computing and online cloud services (also known as cloud computing services) like Infrastructure-as-a-Service (IaaS), Platform-as-a-Service (PaaS), and Software-as-a-Service(SaaS). These services bring numerous benefits, including delegated management and more cost-effective business processes. The user engages these companies as you need them and pay only for what you use, while the providers manage their specific services and ensure that they're working effectively.
Compute Centers: A Compute center is a facility that centralizes an organization's shared IT operations and equipment for the purposes of storing, processing, and disseminating compute power, data and applications. Because they house an organization's most critical and proprietary assets, compute centers are vital to the continuity of daily operations.

Product's Advantages

ANDREAS has the following advantages compared to the current techniques:

- An intelligent scheduler able to optimize resource usage and AI training jobs performance given a power limit set by SysAdmins

- An integrated and tested product
- A skilled support team
- User-friendly product-specific documentation

Product and Support Plan

Product

ANDREAS is combining Machine Learning (ML) techniques with an advanced scheduler and a plug-in for the SLURM batch scheduler. Given a power budget, the ML algorithm, monitoring the system components (CPU, GPU, memory) used by an application, will learn its performance and energy profile. Then, the advanced scheduler will determine: i) the GPUs to assign to each running application, ii) the best job schedule, possibly pre-empting running jobs.
ANDREAS main features are:

- The ability to optimize infrastructure efficiency under power constraints for local cloud or datacenter operators and HPC centres, providing GPU computing power for AI/ML purposes. Local operators/centres are subject to power consumption quotas while their economic results strongly depend on infrastructure usage efficiency. While there are advanced solutions allowing to manage virtual servers or containers, growing ML on GPU demand is the most attractive market to take over from dominating cloud operators but it requires keeping power consumption under hard quotas while optimizing revenue from GPUs which are high-value assets.
- Better power consumption and cost management for innovative SMEs and startups who build their competitive advantage on the AI/ML and need to fit power constraints. Same problems as above apply to entities who build on private clouds to satisfy their needs.
- The ability to manage energy footprint in new infrastructure projects, like distributed GPU computing located at the edge in 5G networks. This is critical for telecommunication companies owning their infrastructure, companies managing others' infrastructure (incl. taking over ownership that is a growing trend in the EU), and SMEs providing innovative solutions.

In all the cases, managing energy footprint and optimizing performance have growing importance to limit carbon footprint and to adopt renewable energy sources which impose stricter constraints.

Intellectual Property Rights

	Item	Owner	IP
Pre-requisites			
	SLURM	E4	background
	ExaMon	E4	foreground
	TF2-gpu on ARM	E4	foreground
	ARMIDA design, manufacturing and system administration	E4	background
	ML models	PoliMi	background
	Pluggable Scheduler	PoliMI	background
	Melodic Process Scheduler	7bulls	background
	Melodic Process Orchestrator	7bulls	background
ANDREAS			
	Algorithms	PoliMI	foreground
	Jobs Manager	7bulls	foreground

Implementation Property Rights

	Open source	Proprietary/owner
SLURM	NO	E4
ExaMon	NO	E4
TF2-gpu on ARM	NO	E4
ARMIDA design, manufacturing and system administration	NO	E4
Algorithms (optimizer in Jobs Manager)	Partially: basic Earliest deadline first released as Apache 2.0 for end user customization, advanced policy based on random greedy NO)	PoliMI
Jobs Manager (queue, manager, db)	NO	7bulls

Pricing Structure

Licensing model for ANDREAS:

	Access to ARMIDA	# of GPU	Support delivery	Support on ANDREAS	Price (EUR)
Flat fee	N/A	License-based	PoliMI, 7bulls: ANDREAS (3Y)	On call (working hours)	50k x 16 GPU (25k ANDREAS + 8k/year support)
ANDREAS-as-a-Service	5 days	As per available in ARMIDA	E4: HW & system SW on ARMIDA PoliMI, 7bulls: ANDREAS	1h of support in 5 days	1,500.00
ANDREAS Bootstrap package	5 days	As per available in ARMIDA	E4: HW & system SW on ARMIDA PoliMI, 7bulls: ANDREAS	10h of support in 5 days	5,000.00

T&C

- Upfront payment
- 6-month accounting period (Jan 1st/June 31st, July 1st/Dec 31st)
- Honor-based

Revenue split

- E4: 40%
- 7bulls: 40%
- PoliMI: 20%

Customer Support/Field Support

Support fees are included in the product's sales price.
Because of the complexity of the possible problems possibly requiring in-depth analysis and code fixes, the partners have agreed to begin the sales process with an achievable level of support (A 'Best Effort' approach will be implemented for the fix of the problems, as well as a 'Best Effort' approach will be implemented for the timeframe for the fixing of the problems) and analyze the number and complexity of the requests for support. Based on the analysis, the proper actions will be taken.
E4 acts as the first point of contact for support and will be dispatching the support calls to PoliMI and 7bulls, who will take charge of the call and connect directly with the customer.
Politecnico di Milano and 7bulls will provide support to users for its own IP (within the Flat fees, ANDREAS-as-a-Service and ANDREAS Bootstrap package license models)

Product Lifecycle

The product has been developed and tested for current-generation technology (mainly NVIDIA GPUs). The GPU vendor has committed to make the future releases of its products backward compatible, in such a way that the product will run also on the new products. E4 will make ARMIDA (the 8-node cluster installed at E4 Computer Engineering SpA) available to PoliMi and to 7bulls for development, testing and duplicating problems (within the ANDREAS-as-a-Service and ANDREAS Bootstrap package license models). Politecnico di Milano and 7bulls have access to ARMIDA for testing and duplicating problems (within the Flat fees, ANDREAS-as-a-Service and ANDREAS Bootstrap package license models. According to the requests of the customers, the partners may further specialize the package, creating customized/enhanced versions of it, which will be released as an update of the previous version

Research and Development

Further development of ANDREAS will be decided according to the success of the first 6 months of sales. As of now, the following areas are under evaluation:

- Optimization in the cloud/multi-cloud environments;
- Additional hardware accelerators optimization (FPGA, etc);

- Optimization of the single GPU usage - bin packing more than one job on the GPU;
- Proactive optimization based on the anticipated workload and GPUs usage.

PoliMI, 7bulls and E4 have the right to publish (separately or with the partners) the results, without prejudice of the background and foreground of the other parties.

Marketing & Sales

Go To Market Strategy

- Identify one or more users/organizations requiring the optimization of the infrastructural cost of ML training jobs. The cloud providers and the data centers are the most suited institutions.
- Analyze the requirements and present the value proposition of ANDREAS both for the users as well as for the data center.
- Identify the optimal license, according to the envisioned requirements.
- Create confidence in the solidity of the product by providing support in the initial phase.
- Collect ROI.

Growth Strategy

To grow the utilization of the product, the following actions will be performed:

- Actively performs marketing campaigns.
- E4 Computer Engineering SpA will make ARMIDA (the 8-node cluster installed at its premises) available to users for running their models (within the ANDREAS-as-a-Service and ANDREAS Bootstrap package license models).
- Participate and present in workshop, congresses, public events.
- Advertise via social media.

Communicate with the Customer

The partners will communicate with its potential customers by:

- Providing product information via email newsletter
- Using targeted Google and Facebook posting
- Utilizing social media such as Twitter, YouTube, Facebook, LinkedIn.
- Providing contact information on the company's website.

Sales channels

E4: Sells licenses
7bulls: Sells licenses
PoliMI: Actively performs influencing (soft power) campaigns

Forecast (as of March 2021)

- New jobs generated: 4 (1 at PoliMI, 2 at E4, 1 at 7BULLS)
- Products improved: 2 (1 for E4, 1 for 7bulls)
- Potential users/customers (estimated): about 10 per year
- Revenue increase (estimated): 150/200K EUR per year for E4 and 7bulls
- Profit increase (estimated): 30/40K EUR per year for E4 and 7bulls

End of Business Plan